

Aula 1 — Fundamentos & Panorama Mobile/Multiplataforma

"Qual stack escolher? Sem critério, vira religião."

Prof. Jackson Smith Moisés Matias

PUC Minas IEC · Arquitetura de Software Distribuído · 21/09/2026

Agenda da aula (alvo ~3h00, buffer até 3h30)

1. **Bloco 1 — Teoria (55min)** — Decisão arquitetural mobile em 2026 + ADRs
2. **Intervalo (30min)**
3. **Bloco 2 — Setup + Hands-on (80min)** — Ambiente por SO + scaffolding RN com IA
4. **Esquenta (15min)** — Atividade 1 ADR + leituras + Q&A

Cronograma é **alvo**, não cronômetro. Acabar 30min antes é vitória, não problema. Buffer reservado pra troubleshoot de setup (Watchman/Pods/AVD travam alunos sempre).

Objetivos de aprendizagem

Ao final da aula você será capaz de:

- **Compreender** taxonomia de apps móveis (nativa, híbrida, PWA, cross-platform)
- **Avaliar** trade-offs entre stacks com base em critérios técnicos e de negócio
- **Aplicar** Architecture Decision Records (ADRs) para documentar escolhas
- **Configurar** ambiente RN (Expo + bare workflow) com IA assistida
- **Identificar** quando ir nativo vs cross-platform com base em literatura acadêmica

Logística da disciplina — avaliação

Notas de **0 a 100**, distribuídas nas 5 atividades + Projeto Final (ver `AVALIACA0.md` do repo).

Faixa	Situação
≥ 70	Aprovado
50 a 69	Exame especial
≤ 49	Reprovado

Exame especial é uma reavaliação adicional pra quem fica na faixa 50-69 — combinamos formato mais perto do fim do curso. Não é reprovação automática, mas também não é aprovação direta.

Por que essa aula importa

| **3 milhões+ de aplicativos** nas lojas. Maioria abandonada após o primeiro uso.

Em 2026, decidir stack mobile é decisão de **5–7 anos de horizonte**. Erro custa caro:

- Migração de stack inteira (Airbnb, 2018)
- Time-to-market estourado (cross-platform mal escolhido)
- Talent gap (não acha gente pra manter)
- Performance debt em hot path

A diferença entre senior e arquiteto é **defender escolha com critério**.

Os 4 tipos fundamentais de apps móveis

Tipo	Exemplo	Quando faz sentido
Nativo	Instagram (iOS), apps bancários	Performance crítica, sensores avançados, talent disponível
Híbrido (WebView)	Apps internos legados, prototipagem	Reuso de stack web existente, complexidade baixa
PWA	Twitter Lite, Starbucks, Pinterest Lite	Web-first, offline robusto, distribuição sem store
Cross-platform	Discord (RN), Coinbase, Alibaba (Flutter)	Time multidisciplinar, ROI cross-OS, lifecycle médio-longo

Fonte: Charland & Leroux (CACM 2011, atualizado 2026)

Trade-off matrix arquitetural

	Nativo	RN	Flutter	KMP	PWA
Performance	9	7	8	8	6
UX nativo	9	8	7	8	6
Cross-OS	3	9	9	8	9
Time-to-market	5	8	8	7	9
Talent pool	9	9	7	5	9
Manutenção	8	7	8	6	8
Tooling maturity	9	9	8	7	8

Pesos variam por contexto. Esta matriz é **ponto de partida**, não resposta universal.

Estudos de caso (sem religião, com fato)

Airbnb (2018): saiu do RN após 2 anos. Razões: complexidade de bridging, talent gap, manutenção dual.

| Post-mortem em 5 partes — leitura obrigatória pré-Aula 3.

Discord (2018+): mantém RN em iOS, nativo em Android. Decisão pragmática.

Coinbase (2018+): unificou em RN. Justificou ganho de velocidade.

Alibaba Xianyu: Flutter pra MAU enorme.

Square / JetBrains: KMP em produção desde 2022.

Shopify (set/2026): saiu do RN após 5 anos (tinha reafirmado o RN em 2025!). Reescreveu o Shop app em Swift/Kotlin — 6 engenheiros, 12 semanas. Razão: IA baixou tanto o custo de manter nativo que o benefício da camada de abstração não compensa mais.

| Fonte: shopify.engineering/back-to-native (post oficial, 10/09/2026)

| Mesmo problema → decisões opostas → ambas defensáveis.

Padrões arquiteturais aplicáveis a mobile

- **Clean Architecture** (Uncle Bob) — separação por camadas
- **MVVM** — clássico em iOS (UIKit/SwiftUI) e Android (Jetpack)
- **MVI** — Model-View-Intent, popular em KMP/Compose
- **TCA** (The Composable Architecture) — iOS/Swift centric
- **Redux/Flux** — base do RTK Query e Zustand

Padrão **não é dogma**. Escolher por aderência ao time + complexidade do app + tooling.

Architecture Decision Records (ADRs)

Padrão proposto por **Michael Nygard (2011)**. Documento curto que registra:

1. **Título** — número + tema
2. **Status** — proposto, aceito, depreciado, substituído
3. **Contexto** — forças em jogo, restrições
4. **Decisão** — escolha feita
5. **Consequências** — efeitos positivos e negativos

ADR é o **artefato** que distingue decisão técnica de palpite.

ADR — exemplo real (banco fictício)

```
# ADR-0042: Stack mobile para app de investimento retail

## Status
Aceito (2026-04-15)

## Contexto
- 5M usuários ativos, 60% iOS / 40% Android
- Compliance BACEN exige biometria + SSL pinning
- Time atual: 12 engenheiros (5 RN, 4 nativos, 3 backend)
- Janela: launch em 9 meses

## Decisão
React Native como base, com módulos nativos (Kotlin + Swift)
para crypto e biometria.

## Consequências
+ Reuso de 70% do codebase
+ Talent disponível
- Bridging nativo exige 2 engs sêniores
- Performance crypto exige profile contínuo
```

ADR — anti-padrões a evitar

- ✗ "ADR depois do código" — vira justificativa, não decisão
- ✗ "ADR de 20 páginas" — ninguém lê
- ✗ "ADR sem alternativas analisadas" — não dá pra criticar
- ✗ "ADR aprovado sem stakeholder técnico" — não tem peso
- ✗ "ADR sem data nem autor" — irracional

Bom ADR cabe em 1 página e mostra o **raciocínio**, não só o resultado.

Estado do mercado mobile 2026

Stack Overflow Survey 2025 + JetBrains Developer Survey 2025:

- React Native: ~38% dos devs mobile
- Flutter: ~32%
- Native Android: ~22%
- Native iOS: ~18%
- KMP: ~9% (crescendo rápido)
- PWA: ~28% (sobreposto com web devs)

| Soma > 100% porque devs usam múltiplas stacks.

Intervalo — 30min

Volta às :____

Aproveita pra:

- Esticar perna
- Verificar Node, JDK, Android Studio instalados
- Tomar café / água

Próximo bloco é hands-on. Vale a pena já abrir terminal + Android Studio.

Setup ambiente — pré-requisitos (comum a todos os SOs)

- **Node 22 LTS** via `nvm` (Windows: `nvm-windows` ou `fnm`)
- **Watchman** (acelera file watching no Metro bundler)
- **Android Studio + AVD** (Pixel 8 API 34) — Linux/Windows/macOS
- **JDK 17** (Android Gradle Plugin 8.x exige)
- **iOS Simulator + Xcode 16+** — **somente macOS** (restrição Apple)
- **Cocoapods** — somente macOS (gerência pods iOS)

Linux/Windows = dev Android + RN web preview. Pra build/test iOS: macOS, cloud Mac (MacStadium, AWS EC2 Mac) ou EAS Build.

Setup ambiente — macOS

```
# Homebrew (se não tiver)
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

# Node 22 via nvm
brew install nvm && nvm install 22 && nvm use 22

# Watchman + Cocoapods + JDK 17
brew install watchman cocoapods openjdk@17

# Android Studio + Xcode
brew install --cask android-studio
# Xcode: instalar pela App Store

# Verificar
node --version           # v22.x
watchman --version       # 2024.x+
xcodebuild -version      # 16.x
adb --version            # 35.x+
```

Setup ambiente — Linux (Ubuntu/Debian)

```
# Node 22 via nvm
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
source ~/.bashrc && nvm install 22 && nvm use 22

# Watchman (build from source ou snap)
sudo apt install -y watchman || sudo snap install watchman

# JDK 17
sudo apt install -y openjdk-17-jdk

# Android Studio: baixar tarball developer.android.com/studio
# Habilitar KVM pro emulador
sudo apt install -y qemu-kvm && sudo adduser $USER kvm

# Verificar
node --version          # v22.x
watchman --version      # 2024.x+
adb --version           # 35.x+
java -version           # 17.x
```

iOS dev no Linux **não suportado**. Use EAS Build (`eas build -p ios`) ou cloud Mac.

Setup ambiente — Windows (PowerShell + WSL2)

```
# Opção A — nativo (recomendado pra Android dev)
# Chocolatey
Set-ExecutionPolicy Bypass -Scope Process -Force
iwr https://community.chocolatey.org/install.ps1 -UseBasicParsing | iex

# Node 22 via nvm-windows
choco install nvm -y
nvm install 22 && nvm use 22

# JDK 17 + Android Studio
choco install temurin17 androidstudio -y

# Verificar (PowerShell)
node --version
adb --version
java -version
```

```
# Opção B — WSL2 (Ubuntu): seguir bloco Linux acima
# Atenção: emulador Android roda melhor no Windows nativo
# (acesso direto ao HAXM/Hyper-V). Code dentro do WSL2 ok.
```

iOS dev no Windows **não suportado**. Use EAS Build ou cloud Mac. Watchman no Windows: opcional, Metro usa

Setup ambiente — Expo SDK

Doc oficial: <https://docs.expo.dev/get-started/set-up-your-environment/>

```
# Guard: Node >= 20 obrigatório (create-expo-app@4 usa Web API File)
node -v | grep -qE 'v(2[0-9]|[3-9][0-9])' \
  || { echo "ERR0: precisa Node 20+, atual $(node -v). Rode: nvm use 22"; exit 1; }

# Quick start (recomendado pela doc — template default c/ Expo Router + TS)
npx create-expo-app@latest meu-app

# Variante: starter mínimo sem Expo Router (didático nesta aula)
npx create-expo-app@latest meu-app --template blank-typescript

cd meu-app && npx expo start
# 'i' iOS Simulator · 'a' Android Emulator · 'w' Web · QR = Expo Go
```

Expo SDK 55+ traz New Architecture habilitada por default.

Pitfall: Node 18 default → `ReferenceError: File is not defined`. Sempre `nvm use 22`.

Fonte da verdade do setup — doc oficial Expo

<https://docs.expo.dev/get-started/set-up-your-environment/>

Esses slides são **snapshot** do que funcionava em **maio/2026** (Expo SDK 55, Node 22).

Regra: se algo divergir entre slide e doc, **a doc oficial vence**.

Expo muda ferramentas e fluxo com frequência (Expo Go iOS, EAS, dev client, prebuild). Quem segue a doc não fica pra trás.

Quando consultar:

- Mudou versão do SDK? → doc tem migration guide
- Comando não funciona? → doc oficial primeiro, Stack Overflow depois
- Quer recurso novo (Expo Router, EAS, dev client)? → doc tem walkthrough

Bookmark essa URL **agora**. Vai usar muito ao longo do semestre.

Onde rodar o app — opções (importante!)

Opção	Custo	Plataforma	Notas
Expo Go (Android)	grátis	Play Store	Padrão didático. Funciona.
Expo Go (iOS)	\$99/ano	Apple Dev Program + TestFlight	Mudou em 2025+ — não recomendo pra aula
iOS Simulator	grátis	macOS + Xcode 16+	Mais confiável que Expo Go iOS
Android Emulator	grátis	Android Studio + AVD	Bom, exige RAM (>=16GB host)
Web (expo start --web)	grátis	qualquer browser	Backup universal, sem device
Snack (snack.expo.dev)	grátis	browser	Sem instalar nada, preview live

Recomendação aula 1: Android Expo Go OU Simulator iOS (macOS) OU Snack/Web. Não exija Apple Dev Program de aluno.

IA assistida — scaffolding (Claude Code e alternativas)

Cursor, Claude Code, Gemini CLI são produtividade real, não brinquedo.

Prompt útil (funciona em qualquer LLM):

```
Estou criando app RN com Expo SDK 55.  
Crie estrutura inicial com:  
- Clean Architecture (data/domain/presentation)  
- React Navigation v7 (stack + tabs)  
- Redux Toolkit + RTK Query  
- TypeScript strict mode  
  
Liste arquivos e gere scaffolding.
```

Use IA para **transformação** (boilerplate, scaffolding). Resista a usá-la para **julgamento** (decisão arquitetural).

IA assistida — escolher ferramenta (free vs pago)

Ferramenta	Free?	UX	Notas
Claude Code CLI	Não real (Pro \$20/mês limitado)	Terminal	Padrão dessas aulas
Gemini CLI	Sim, free (open source + Code Assist)	Terminal	Alternativa mais próxima de Claude Code
Cursor	Free tier (2000 completions/mês)	IDE (fork VS Code)	Inline edit + chat
GitHub Copilot	Free estudantes/OSS (cadastra Education)	IDE plugin	Autocomplete forte
ChatGPT (browser)	Sim (GPT-4o-mini ilimitado; 5/4.1 limitado)	Browser	Copy-paste prompt
Gemini (browser)	Sim (gemini.google.com, 2.5 Pro limitado)	Browser	Alternativa browser
Codex CLI (OpenAI)	Requer ChatGPT Plus (\$20/mês)	Terminal	Pago

Sem orçamento? Caminho recomendado: **Gemini CLI** (terminal, free) + **Cursor free** (IDE). Cobrem 90% do que Claude Code faz nesta aula.

Free tier não cobre uso pesado em produção. Pra trabalho diário, ferramenta paga compensa fácil.

Hands-on — vamos fazer juntos (~55min + 10min discussão)

Roteiro realista (primeiro contato — [rodar via Web](#))

1. `npx create-expo-app meu-app --template blank-typescript` (5min)
2. `cd meu-app && npx expo start --web` — abre no browser (5min)
| Web = zero device, zero simulator, zero Apple Dev Program. Padrão de aula 1.
3. Editar `App.tsx` + criar 2ª tela (10min)
4. Instalar React Navigation v7 + Stack (10min)
5. Estado com Redux Toolkit — slice simples (10min)
6. Inspecionar via **React DevTools (browser)** (5min) — Flipper deprecated em RN 0.74+
7. **(Bônus)** quem quiser: rodar em Android Expo Go OU iOS Simulator (10min)
8. Discussão + dúvidas (10min)

Skip se travar: Clean Architecture full + TypeScript strict — vão pra Aula 2.

Por que web? APIs específicas de device (sensores, câmera, push) ficam pra Aula 2/3. Em Aula 1 importa **fluxo de dev** (Metro, hot reload, navegação, estado).

Repo starter: github.com/jacksonsmith/puc-iec-mobile-multiplataforma/tree/main/starters/aula-01

Pitfalls comuns (aprenda com meus erros)

- ✗ **Node 18 default** → `ReferenceError: File is not defined` no `create-expo-app`. Sempre `nvm use 22` (ou `nvm alias default 22`)
- ✗ **Esquecer de instalar Watchman** — hot reload quebra de forma inexplicável
- ✗ **iOS Simulator desatualizado** — Xcode pede update no meio da aula
- ✗ **Pods desatualizados** — `cd ios && pod install` salva muito tempo
- ✗ **Variáveis de ambiente não carregadas** — `.env` precisa do plugin Expo
- ✗ **TypeScript strict ignorado** — bugs que apareceriam em build aparecem em produção

Dica: tenha 1 environment-check script em `scripts/check-env.sh` que valida tudo antes da aula (Node $\geq$ 20, watchman, JDK, adb).

Atividade assíncrona — Atividade 1: ADR Arquitetural (15 pts)

Entrega: ver Canvas.

Tarefa:

1. Escolher caso real (app bancário, e-commerce, mídia, saúde)
2. Redigir ADR completo justificando escolha entre nativo / RN / Flutter / KMP / PWA
3. **Matriz quantitativa** — tabela com nota (1-10) × peso por critério, score ponderado por alternativa (≥ 4 alternativas)
4. **≥ 3 fontes confiáveis**, sendo **≥ 1 acadêmica** (paper peer-reviewed ou livro técnico). Outras 2: doc oficial (RN/Flutter/KMP), post de engenharia validado (Airbnb/Discord/Netflix Tech Blog), white paper

Enunciado completo + critérios + template ADR:

github.com/jacksonsmith/puc-iec-mobile-multiplataforma/tree/main/exercicios/01-adr-arquitetural

Template ADR direto: [exercicios/adr-template.md](#)

Entrega: fork do repo + PR + upload no Canvas.

Leituras obrigatórias (pré-Aula 2)

- Charland, A.; Leroux, B. (2011) *Mobile Application Development: Web vs. Native*. ACM Queue 9(4) / CACM 54(5).
- **React Native docs** (oficial) — *Getting Started + Core Components + React Fundamentals*.
<https://reactnative.dev/docs/getting-started>
- Nygard, M. (2011) *Documenting Architecture Decisions*.
- Peal, G. / **Airbnb Engineering** (2018) *Sunsetting React Native* — série de 5 posts.

PDFs em `material-de-apoio/aula-01/` (todas fontes públicas).

Quem não ler **fica perdido** na Aula 2 (RN deep dive).

Recap da aula

- Apps móveis em 4 categorias (nativo, híbrido, PWA, cross-platform) com trade-offs distintos
- Decisão arquitetural exige ADR — não vira religião
- Cases Airbnb, Discord, Coinbase, Shopify mostram que **mesma situação → decisões opostas defensáveis** (Shopify até reverteu a própria decisão em 2026)
- Padrões aplicáveis: Clean, MVVM, MVI, TCA, Redux/Flux
- Setup RN + IA assistida funcionando

Próxima aula (28/09): RN New Architecture — JSI, Fabric, TurboModules, Hermes.

Obrigado!

Dúvidas até a Aula 2 → Teams

 jackson.96@gmail.com

 linkedin.com/in/3jacksonsmith

 github.com/jacksonsmith

Próxima segunda, 28/09, 19:00